

Distributed memory programming with MPI

3

Recall that the world of parallel multiple instruction, multiple data, or MIMD, computers is, for the most part, divided into **distributed-memory** and **shared-memory** systems. From a programmer's point of view, a distributed-memory system consists of a collection of core-memory pairs connected by a network, and the memory associated with a core is directly accessible only to that core. (See Fig. 3.1.) On the other hand, from a programmer's point of view, a shared-memory system consists of a collection of cores connected to a globally accessible memory, in which each core can have access to any memory location. (See Fig. 3.2.) In this chapter, we're going to start looking at how to program distributed memory systems using **message-passing**.

Recall that in message-passing programs, a program running on one core-memory pair is usually called a **process**, and two processes can communicate by calling functions: one process calls a *send* function and the other calls a *receive* function. The implementation of message-passing that we'll be using is called **MPI**, which is an abbreviation of **Message-Passing Interface**. MPI is not a new programming language. It defines a *library* of functions that can be called from C and Fortran programs. We'll learn about some of MPI's different send and receive functions. We'll also

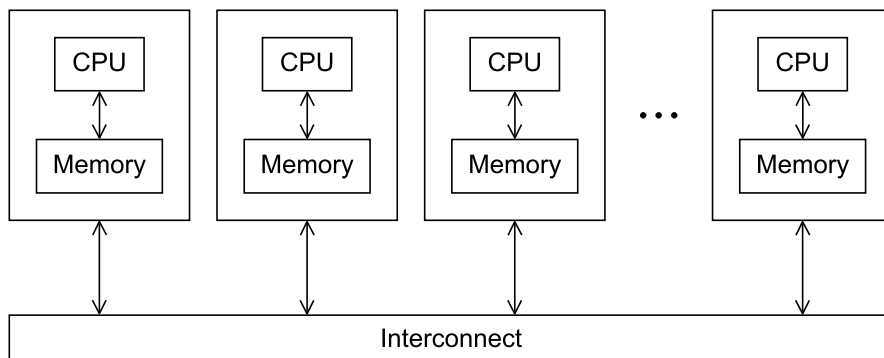
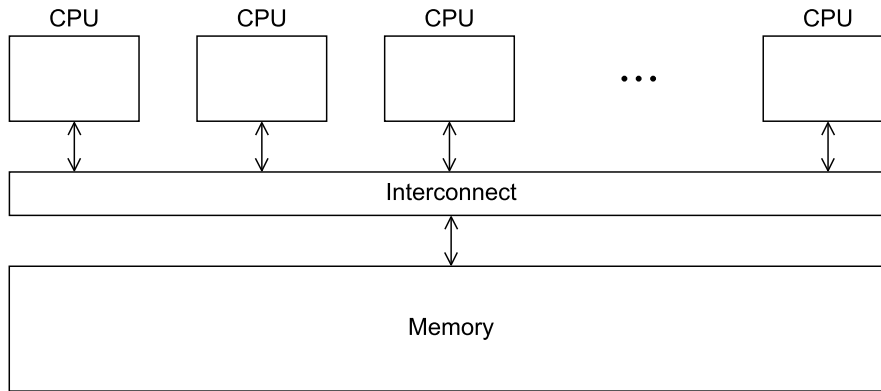


FIGURE 3.1

A distributed memory system.

**FIGURE 3.2**

A shared memory system.

learn about some “global” communication functions that can involve more than two processes. These functions are called **collective** communications. In the process of learning about all of these MPI functions, we’ll also learn about some of the fundamental issues involved in writing message-passing programs—issues such as data partitioning and I/O in distributed-memory systems. We’ll also revisit the issue of parallel program performance.

3.1 Getting started

Perhaps the first program that many of us saw was some variant of the “hello, world” program in Kernighan and Ritchie’s classic text [32]:

```
#include <stdio.h>

int main(void) {
    printf("hello, world\n");

    return 0;
}
```

Let’s write a program similar to “hello, world” that makes some use of MPI. Instead of having each process simply print a message, we’ll designate one process to do the output, and the other processes will send it messages, which it will print.

In parallel programming, it’s common (one might say standard) for the processes to be identified by nonnegative integer *ranks*. So if there are p processes, the pro-

cesses will have ranks $0, 1, 2, \dots, p - 1$. For our parallel “hello, world,” let’s make process 0 the designated process, and the other processes will send it messages. See Program 3.1.

```

1 #include <stdio.h>
2 #include <string.h> /* For strlen */
3 #include <mpi.h>    /* For MPI functions , etc */
4
5 const int MAX_STRING = 100;
6
7 int main(void) {
8     char    greeting[MAX_STRING];
9     int     comm_sz; /* Number of processes */
10    int     my_rank; /* My process rank */
11
12    MPI_Init(NULL, NULL);
13    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
14    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
15
16    if (my_rank != 0) {
17        sprintf(greeting, "Greetings from process %d of %d!",
18                my_rank, comm_sz);
19        MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
20                MPI_COMM_WORLD);
21    } else {
22        printf("Greetings from process %d of %d!\n",
23                my_rank, comm_sz);
24        for (int q = 1; q < comm_sz; q++) {
25            MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
26                    0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
27            printf("%s\n", greeting);
28        }
29    }
30
31    MPI_Finalize();
32    return 0;
33 } /* main */

```

Program 3.1: MPI program that prints greetings from the processes.

3.1.1 Compilation and execution

The details of compiling and running the program depend on your system, so you may need to check with a local expert. However, recall that when we need to be explicit, we’ll assume that we’re using a text editor to write the program source, and

the command line to compile and run. Many systems use a command called `mpicc` for compilation¹:

```
$ mpicc -g -Wall -o mpi_hello mpi_hello.c
```

Typically `mpicc` is a script that's a **wrapper** for the C compiler. A **wrapper script** is a script whose main purpose is to run some program. In this case, the program is the C compiler. However, the wrapper simplifies the running of the compiler by telling it where to find the necessary header files, and which libraries to link with the object file.

Many systems also support program startup with `mpiexec`:

```
$ mpiexec -n <number of processes> ./mpi_hello
```

So to run the program with one process, we'd type

```
$ mpiexec -n 1 ./mpi_hello
```

and to run the program with four processes, we'd type

```
$ mpiexec -n 4 ./mpi_hello
```

With one process, the program's output would be

```
Greetings from process 0 of 1!
```

and with four processes, the program's output would be

```
Greetings from process 0 of 4!
Greetings from process 1 of 4!
Greetings from process 2 of 4!
Greetings from process 3 of 4!
```

How do we get from invoking `mpiexec` to one or more lines of greetings? The `mpiexec` command tells the system to start `<number of processes>` instances of our `<mpi_hello>` program. It may also tell the system which core should run each instance of the program. After the processes are running, the MPI implementation takes care of making sure that the processes can communicate with each other.

3.1.2 MPI programs

Let's take a closer look at the program.

The first thing to observe is that this *is* a C program. For example, it includes the standard C header files `stdio.h` and `string.h`. It also has a `main` function, just like any other C program. However, there are many parts of the program that are new. Line 3 includes the `mpi.h` header file. This contains prototypes of MPI functions, macro definitions, type definitions, and so on; it contains all the definitions and declarations needed for compiling an MPI program.

¹ Recall that the dollar sign (\$) is the shell prompt—so it shouldn't be typed in. Also recall that, for the sake of explicitness, we assume that we're using the Gnu C compiler, `gcc`, and we always use the options `-g`, `-Wall`, and `-o`. See Section 2.9 for further information.

The second thing to observe is that all of the identifiers defined by MPI start with the string `MPI_`. The first letter following the underscore is capitalized for function names and MPI-defined types. All of the letters in MPI-defined macros and constants are capitalized. So there's no question about what is defined by MPI and what's defined by the user program.

3.1.3 `MPI_Init` and `MPI_Finalize`

In Line 12, the call to `MPI_Init` tells the MPI system to do all of the necessary setup. For example, it might allocate storage for message buffers, and it might decide which process gets which rank. As a rule of thumb, no other MPI functions should be called before the program calls `MPI_Init`. Its syntax is

```
int MPI_Init(
    int*      argc_p    /* in/out */,
    char***   argv_p    /* in/out */);
```

The arguments, `argc_p` and `argv_p`, are pointers to the arguments to `main`, `argc` and `argv`. However, when our program doesn't use these arguments, we can just pass `NULL` for both. Like most MPI functions, `MPI_Init` returns an `int` error code. In most cases, we'll ignore the error codes, since checking them tends to clutter the code and make it more difficult to understand what it's doing.²

In Line 31, the call to `MPI_Finalize` tells the MPI system that we're done using MPI, and that any resources allocated for MPI can be freed. The syntax is quite simple:

```
int MPI_Finalize(void);
```

In general, no MPI functions should be called after the call to `MPI_Finalize`.

Thus a typical MPI program has the following basic outline:

```
. . .
#include <mpi.h>
. . .
int main(int argc, char* argv[]) {
    . . .
    /* No MPI calls before this */
    MPI_Init(&argc, &argv);
    . . .
    MPI_Finalize();
    /* No MPI calls after this */
    . . .
    return 0;
}
```

² Of course, when we're debugging our programs, we may make extensive use of the MPI error codes.

However, we’ve already seen that it’s not necessary to pass pointers to `argc` and `argv` to `MPI_Init`. It’s also not necessary that the calls to `MPI_Init` and `MPI_Finalize` be in `main`.

3.1.4 Communicators, `MPI_Comm_size`, and `MPI_Comm_rank`

In MPI a **communicator** is a collection of processes that can send messages to each other. One of the purposes of `MPI_Init` is to define a communicator that consists of all of the processes started by the user when she started the program. This communicator is called `MPI_COMM_WORLD`. The function calls in Lines 13 and 14 are getting information about `MPI_COMM_WORLD`. Their syntax is

```
int MPI_Comm_size (
    MPI_Comm comm          /* in */,
    int* comm_sz_p         /* out */);

int MPI_Comm_rank (
    MPI_Comm comm          /* in */,
    int* my_rank_p         /* out */);
```

For both functions, the first argument is a communicator and has the special type defined by MPI for communicators, `MPI_Comm`. `MPI_Comm_size` returns in its second argument the number of processes in the communicator, `MPI_Comm_rank` returns in its second argument the calling process’s rank in the communicator. We’ll often use the variable `comm_sz` for the number of processes in `MPI_COMM_WORLD`, and the variable `my_rank` for the process rank.

3.1.5 SPMD programs

Notice that we compiled a single program—we didn’t compile a different program for each process—and we did this in spite of the fact that process 0 is doing something fundamentally different from the other processes: it’s receiving a series of messages and printing them, while each of the other processes is creating and sending a message. This is quite common in parallel programming. In fact, *most* MPI programs are written in this way. That is, a single program is written so that different processes carry out different actions, and this can be achieved by simply having the processes branch on the basis of their process rank. Recall that this approach to parallel programming is called **single program, multiple data** or **SPMD**. The `if–else` statement in Lines 16–29 makes our program SPMD.

Also notice that our program will, in principle, run with any number of processes. We saw a little while ago that it can be run with one process or four processes, but if our system has sufficient resources, we could also run it with 1000 or even 100,000 processes. Although MPI doesn’t require that programs have this property, it’s almost always the case that we try to write programs that will run with any number of processes, because we usually don’t know in advance the exact resources available to us. For example, we might have a 20-core system available today, but tomorrow we might have access to a 500-core system.

3.1.6 Communication

In Lines 17–18, each process, other than process 0, creates a message it will send to process 0. (The function `sprintf` is very similar to `printf`, except that instead of writing to `stdout`, it writes to a string.) Lines 19–20 actually send the message to process 0. Process 0, on the other hand, simply prints its message using `printf`, and then uses a `for` loop to receive and print the messages sent by processes 1, 2, ..., `comm_sz - 1`. Lines 25–26 receive the message sent by process q , for $q = 1, 2, \dots, \text{comm_sz} - 1$.

3.1.7 MPI_Send

The sends executed by processes 1, 2, ..., `comm_sz - 1` are fairly complex, so let's take a closer look at them. Each of the sends is carried out by a call to `MPI_Send`, whose syntax is

```
int MPI_Send(
    void*      msg_buf_p    /* in */,
    int        msg_size     /* in */,
    MPI_Datatype msg_type    /* in */,
    int        dest         /* in */,
    int        tag          /* in */,
    MPI_Comm   communicator /* in */);
```

The first three arguments, `msg_buf_p`, `msg_size`, and `msg_type`, determine the contents of the message. The remaining arguments, `dest`, `tag`, and `communicator`, determine the destination of the message.

The first argument, `msg_buf_p`, is a pointer to the block of memory containing the contents of the message. In our program, this is just the string containing the message, `greeting`. (Remember that in C an array, such as a string, is a pointer.) The second and third arguments, `msg_size` and `msg_type`, determine the amount of data to be sent. In our program, the `msg_size` argument is the number of characters in the message, plus one character for the `'\0'` character that terminates C strings. The `msg_type` argument is `MPI_CHAR`. These two arguments together tell the system that the message contains `strlen(greeting)+1` chars.

Since C types (`int`, `char`, etc.) can't be passed as arguments to functions, MPI defines a special type, `MPI_Datatype`, that is used for the `msg_type` argument. MPI also defines a number of constant values for this type. The ones we'll use (and a few others) are listed in Table 3.1.

Notice that the size of the string `greeting` is not the same as the size of the message specified by the arguments `msg_size` and `msg_type`. For example, when we run the program with four processes, the length of each of the messages is 31 characters, while we've allocated storage for 100 characters in `greetings`. Of course, the size of the message sent should be less than or equal to the amount of storage in the buffer—in our case the string `greeting`.

The fourth argument, `dest`, specifies the rank of the process that should receive the message. The fifth argument, `tag`, is a nonnegative `int`. It can be used to distinguish

Table 3.1 Some predefined MPI datatypes.

MPI datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

messages that are otherwise identical. For example, suppose process 1 is sending **floats** to process 0. Some of the **floats** should be printed, while others should be used in a computation. Then the first four arguments to `MPI_Send` provide no information regarding which **floats** should be printed and which should be used in a computation. So process 1 can use (say) a tag of 0 for the messages that should be printed and a tag of 1 for the messages that should be used in a computation.

The final argument to `MPI_Send` is a communicator. All MPI functions that involve communication have a communicator argument. One of the most important purposes of communicators is to specify communication universes; recall that a communicator is a collection of processes that can send messages to each other. Conversely, a message sent by a process using one communicator cannot be received by a process that's using a different communicator. Since MPI provides functions for creating new communicators, this feature can be used in complex programs to ensure that messages aren't "accidentally received" in the wrong place.

An example will clarify this. Suppose we're studying global climate change, and we've been lucky enough to find two libraries of functions, one for modeling the earth's atmosphere and one for modeling the earth's oceans. Of course, both libraries use MPI. These models were built independently, so they don't communicate with each other, but they do communicate internally. It's our job to write the interface code. One problem we need to solve is ensuring that the messages sent by one library won't be accidentally received by the other. We might be able to work out some scheme with tags: the atmosphere library gets tags $0, 1, \dots, n-1$, and the ocean library gets tags $n, n+1, \dots, n+m$. Then each library can use the given range to figure out which tag it should use for which message. However, a much simpler solution is provided by communicators: we simply pass one communicator to the atmosphere library functions and a different communicator to the ocean library functions.

3.1.8 MPI_Recv

The first six arguments to `MPI_Recv` correspond to the first six arguments of `MPI_Send`:

```
int MPI_Recv(
    void*      msg_buf_p    /* out */,
    int        buf_size     /* in  */,
    MPI_Datatype buf_type    /* in  */,
    int        source       /* in  */,
    int        tag          /* in  */,
    MPI_Comm   communicator /* in  */,
    MPI_Status* status_p    /* out */);
```

Thus the first three arguments specify the memory available for receiving the message: `msg_buf_p` points to the block of memory, `buf_size` determines the number of objects that can be stored in the block, and `buf_type` indicates the type of the objects. The next three arguments identify the message. The `source` argument specifies the process from which the message should be received. The `tag` argument should match the `tag` argument of the message being sent, and the `communicator` argument must match the communicator used by the sending process. We'll talk about the `status_p` argument shortly. In many cases, it won't be used by the calling function, and, as in our "greetings" program, the special MPI constant `MPI_STATUS_IGNORE` can be passed.

3.1.9 Message matching

Suppose process q calls `MPI_Send` with

```
MPI_Send(send_buf_p, send_buf_sz, send_type, dest, send_tag,
         send_comm);
```

Also suppose that process r calls `MPI_Recv` with

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,
         recv_comm, &status);
```

Then the message sent by q with the above call to `MPI_Send` can be received by r with the call to `MPI_Recv` if

- `recv_comm = send_comm`,
- `recv_tag = send_tag`,
- `dest = r`, and
- `src = q`.

These conditions aren't quite enough for the message to be *successfully* received, however. The parameters specified by the first three pairs of arguments, `send_buf_p/recv_buf_p`, `send_buf_sz/recv_buf_sz`, and `send_type/recv_type`, must specify compatible buffers. For detailed rules, see the MPI-3 specification [40]. Most of the time, the following rule will suffice:

- If `recv_type = send_type` and `recv_buf_sz ≥ send_buf_sz`, then the message sent by q can be successfully received by r .

Of course, it can happen that one process is receiving messages from multiple processes, *and* the receiving process doesn't know the order in which the other processes will send the messages. For example, suppose process 0 is doling out work to processes 1, 2, ..., `comm_sz - 1`, and processes 1, 2, ..., `comm_sz - 1`, send their results back to process 0 when they finish the work. If the work assigned to each process takes an unpredictable amount of time, then 0 has no way of knowing the order in which the processes will finish. If process 0 simply receives the results in process rank order—first the results from process 1, then the results from process 2, and so on—and if (say) process `comm_sz-1` finishes first, it *could* happen that process `comm_sz-1` could sit and wait for the other processes to finish. To avoid this problem MPI provides a special constant `MPI_ANY_SOURCE` that can be passed to `MPI_Recv`. Then, if process 0 executes the following code, it can receive the results in the order in which the processes finish:

```
for (i = 1; i < comm_sz; i++) {
    MPI_Recv(result, result_sz, result_type, MPI_ANY_SOURCE,
             result_tag, comm, MPI_STATUS_IGNORE);
    Process_result(result);
}
```

Similarly, it's possible that one process can be receiving multiple messages with different tags from another process, and the receiving process doesn't know the order in which the messages will be sent. For this circumstance, MPI provides the special constant `MPI_ANY_TAG` that can be passed to the `tag` argument of `MPI_Recv`.

A couple of points should be stressed in connection with these “wildcard” arguments:

1. Only a receiver can use a wildcard argument. Senders must specify a process rank and a nonnegative tag. So MPI uses a “push” communication mechanism rather than a “pull” mechanism.
2. There is no wildcard for communicator arguments; both senders and receivers must always specify communicators.

3.1.10 The `status_p` argument

If you think about these rules for a minute, you'll notice that a receiver can receive a message without knowing

1. the amount of data in the message,
2. the sender of the message, or
3. the tag of the message.

So how can the receiver find out these values? Recall that the last argument to `MPI_Recv` has type `MPI_Status*`. The MPI type `MPI_Status` is a struct with at least the three members `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR`. Suppose our program contains

the definition

```
MPI_Status status;
```

Then after a call to `MPI_Recv`, in which `&status` is passed as the last argument, we can determine the sender and tags by examining the two members:

```
status.MPI_SOURCE
status.MPI_TAG
```

The amount of data that's been received isn't stored in a field that's directly accessible to the application program. However, it can be retrieved with a call to `MPI_Get_count`. For example, suppose that in our call to `MPI_Recv`, the type of the receive buffer is `recv_type` and, once again, we passed in `&status`. Then the call

```
MPI_Get_count(&status, recv_type, &count)
```

will return the number of elements received in the `count` argument. In general, the syntax of `MPI_Get_count` is

```
int MPI_Get_count(
    MPI_Status* status_p /* in */,
    MPI_Datatype type /* in */,
    int* count_p /* out */);
```

Note that the count isn't directly accessible as a member of the `MPI_Status` variable, simply because it depends on the type of the received data, and, consequently, determining it would probably require a calculation (e.g., (number of bytes received)/(bytes per object)). And if this information isn't needed, we shouldn't waste a calculation determining it.

3.1.11 Semantics of `MPI_Send` and `MPI_Recv`

What exactly happens when we send a message from one process to another? Many of the details depend on the particular system, but we can make a few generalizations. The sending process will assemble the message. For example, it will add the “envelope” information to the actual data being transmitted—the destination process rank, the sending process rank, the tag, the communicator, and some information on the size of the message. Once the message has been assembled, recall from Chapter 2 that there are essentially two possibilities: the sending process can **buffer** the message or it can **block**. If it buffers the message, the MPI system will place the message (data and envelope) into its own internal storage, and the call to `MPI_Send` will return.

Alternatively, if the system blocks, it will wait until it can begin transmitting the message, and the call to `MPI_Send` may not return immediately. Thus if we use `MPI_Send`, when the function returns, we don't actually know whether the message has been transmitted. We only know that the storage we used for the message, the send buffer, is available for reuse by our program. If we need to know that the message has been transmitted, or if we need for our call to `MPI_Send` to return immediately—regardless of whether the message has been sent—MPI provides alternative functions for sending. We'll learn about one of these alternative functions later.

The exact behavior of `MPI_Send` is determined by the MPI implementation. However, typical implementations have a default “cutoff” message size. If the size of a message is less than the cutoff, it will be buffered. If the size of the message is greater than the cutoff, `MPI_Send` will block.

Unlike `MPI_Send`, `MPI_Recv` always blocks until a matching message has been received. So when a call to `MPI_Recv` returns, we know that there is a message stored in the receive buffer (unless there’s been an error). There is an alternate method for receiving a message, in which the system checks whether a matching message is available and returns, regardless of whether there is one.

MPI requires that messages be *non-overtaking*. This means that if process q sends two messages to process r , then the first message sent by q must be available to r before the second message. However, there is no restriction on the arrival of messages sent from different processes. That is, if q and t both send messages to r , then even if q sends its message before t sends its message, there is no requirement that q ’s message become available to r before t ’s message. This is essentially because MPI can’t impose performance on a network. For example, if q happens to be running on a machine on Mars, while r and t are both running on the same machine in San Francisco, and if q sends its message a nanosecond before t sends its message, it would be extremely unreasonable to require that q ’s message arrive before t ’s.

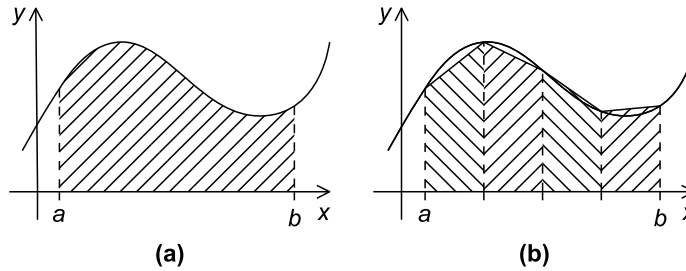
3.1.12 Some potential pitfalls

Note that the semantics of `MPI_Recv` suggests a potential pitfall in MPI programming: If a process tries to receive a message and there’s no matching send, then the process will block forever. That is, the process will **hang**. So when we design our programs, we need to be sure that every receive has a matching send. Perhaps even more important, we need to be very careful when we’re coding that there are no inadvertent mistakes in our calls to `MPI_Send` and `MPI_Recv`. For example, if the tags don’t match, or if the rank of the destination process is the same as the rank of the source process, the receive won’t match the send, and either a process will hang, or, perhaps worse, the receive may match *another* send.

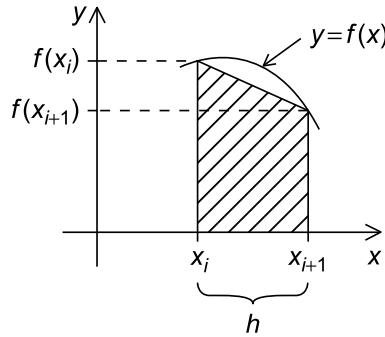
Similarly, if a call to `MPI_Send` blocks and there’s no matching receive, then the sending process can hang. If, on the other hand, a call to `MPI_Send` is buffered and there’s no matching receive, then the message will be lost.

3.2 The trapezoidal rule in MPI

Printing messages from processes is all well and good, but we’re probably not taking the trouble to learn to write MPI programs just to print messages. So let’s take a look at a somewhat more useful program—let’s write a program that implements the trapezoidal rule for numerical integration.

**FIGURE 3.3**

The trapezoidal rule: (a) area to be estimated and (b) approximate area using trapezoids.

**FIGURE 3.4**

One trapezoid.

3.2.1 The trapezoidal rule

Recall that we can use the trapezoidal rule to approximate the area between the graph of a function, $y = f(x)$, two vertical lines, and the x -axis. (See Fig. 3.3.) The basic idea is to divide the interval on the x -axis into n equal subintervals. Then we approximate the area lying between the graph and each subinterval by a trapezoid, whose base is the subinterval, whose vertical sides are the vertical lines through the endpoints of the subinterval, and whose fourth side is the secant line joining the points where the vertical lines cross the graph. (See Fig. 3.4.) If the endpoints of the subinterval are x_i and x_{i+1} , then the length of the subinterval is $h = x_{i+1} - x_i$. Also, if the lengths of the two vertical segments are $f(x_i)$ and $f(x_{i+1})$, then the area of the trapezoid is

$$\text{Area of one trapezoid} = \frac{h}{2}[f(x_i) + f(x_{i+1})].$$

Since we chose the n subintervals so that they would all have the same length, we also know that if the vertical lines bounding the region are $x = a$ and $x = b$, then

$$h = \frac{b-a}{n}.$$

Thus if we call the leftmost endpoint x_0 , and the rightmost endpoint x_n , we have that

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_{n-1} = a + (n-1)h, x_n = b,$$

and the sum of the areas of the trapezoids—our approximation to the total area—is

$$\text{Sum of trapezoid areas} = h[f(x_0)/2 + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + f(x_n)/2].$$

Thus, pseudocode for a serial program might look something like this:

```
/* Input: a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
    x_i = a + i*h;
    approx += f(x_i);
}
approx = h*approx;
```

3.2.2 Parallelizing the trapezoidal rule

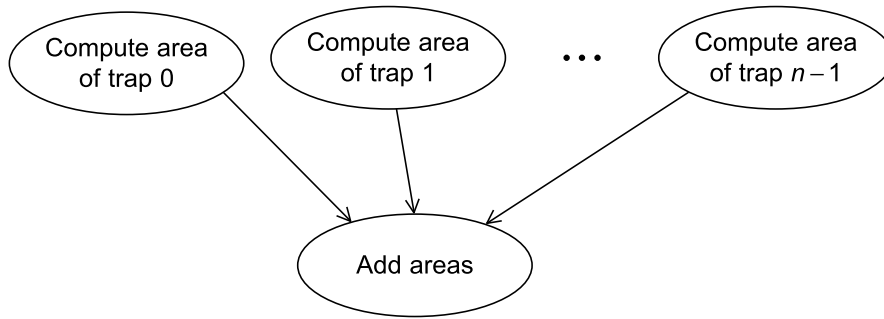
It is not the most attractive word, but, as we noted in Chapter 1, people who write parallel programs do use the verb “parallelize” to describe the process of converting a serial program or algorithm into a parallel program.

Recall that we can design a parallel program by using four basic steps:

1. Partition the problem solution into tasks.
2. Identify communication channels between the tasks.
3. Aggregate tasks into composite tasks.
4. Map composite tasks to cores.

In the partitioning phase, we usually try to identify as many tasks as possible. For the trapezoidal rule, we might identify two types of tasks: one type is finding the area of a single trapezoid, and the other is computing the sum of these areas. Then the communication channels will join each of the tasks of the first type to the single task of the second type. (See Fig. 3.5.)

So how can we aggregate the tasks and map them to the cores? Our intuition tells us that the more trapezoids we use, the more accurate our estimate will be. That is, we should use many trapezoids, and we will use many more trapezoids than cores. Thus we need to aggregate the computation of the areas of the trapezoids into groups. A natural way to do this is to split the interval $[a, b]$ up into `comm_sz` subintervals. If `comm_sz` evenly divides n , the number of trapezoids, we can simply apply the trapezoidal rule with $n/\text{comm_sz}$ trapezoids to each of the `comm_sz` subintervals. To finish, we can have one of the processes, say process 0, add the estimates.

**FIGURE 3.5**

Tasks and communications for the trapezoidal rule.

Let's make the simplifying assumption that `comm_sz` evenly divides n . Then pseudocode for the program might look something like the following:

```

1   Get a, b, n;
2   h = (b-a)/n;
3   local_n = n/comm_sz;
4   local_a = a + my_rank*local_n*h;
5   local_b = local_a + local_n*h;
6   local_integral = Trap(local_a, local_b, local_n, h);
7   if (my_rank != 0)
8       Send local_integral to process 0;
9   else /* my_rank == 0 */
10      total_integral = local_integral;
11      for (proc = 1; proc < comm_sz; proc++) {
12          Receive local_integral from proc;
13          total_integral += local_integral;
14      }
15  }
16  if (my_rank == 0)
17      print result;

```

Let's defer, for the moment, the issue of input and just “hardwire” the values for a , b , and n . When we do this, we get the MPI program shown in Program 3.2. The `Trap` function is just an implementation of the serial trapezoidal rule. See Program 3.3.

Notice that in our choice of identifiers, we try to differentiate between *local* and *global* variables. **Local** variables are variables whose contents are significant only on the process that's using them. Some examples from the trapezoidal rule program are `local_a`, `local_b`, and `local_n`. Variables whose contents are significant to all the processes are sometimes called **global** variables. Some examples from the trapezoidal rule are `a`, `b`, and `n`. Note that this usage is different from the usage you learned in your introductory programming class, where local variables are private to a single function and global variables are accessible to all the functions. However, no confusion should arise, since the context should make the meaning clear.

```

1  int main(void) {
2      int my_rank, comm_sz, n = 1024, local_n;
3      double a = 0.0, b = 3.0, h, local_a, local_b;
4      double local_int, total_int;
5      int source;
6
7      MPI_Init(NULL, NULL);
8      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
9      MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
10
11     h = (b-a)/n;          /* h is the same for all processes */
12     local_n = n/comm_sz; /* So is the number of trapezoids */
13
14     local_a = a + my_rank*local_n*h;
15     local_b = local_a + local_n*h;
16     local_int = Trap(local_a, local_b, local_n, h);
17
18     if (my_rank != 0) {
19         MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
20                 MPI_COMM_WORLD);
21     } else {
22         total_int = local_int;
23         for (source = 1; source < comm_sz; source++) {
24             MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
25                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26             total_int += local_int;
27         }
28     }
29
30     if (my_rank == 0) {
31         printf("With n = %d trapezoids, our estimate\n", n);
32         printf("of the integral from %f to %f = %.15e\n",
33               a, b, total_int);
34     }
35     MPI_Finalize();
36     return 0;
37 } /* main */

```

Program 3.2: First version of MPI trapezoidal rule.

3.3 Dealing with I/O

Of course, the current version of the parallel trapezoidal rule has a serious deficiency: it will only compute the integral over the interval $[0, 3]$ using 1024 trapezoids. We can edit the code and recompile, but this is quite a bit of work compared to simply typing in three new numbers. We need to address the problem of getting input from


```

1  double Trap(
2      double left_endpt /* in */,
3      double right_endpt /* in */,
4      int trap_count /* in */,
5      double base_len /* in */) {
6      double estimate, x;
7      int i;
8
9      estimate = (f(left_endpt) + f(right_endpt))/2.0;
10     for (i = 1; i <= trap_count-1; i++) {
11         x = left_endpt + i*base_len;
12         estimate += f(x);
13     }
14     estimate = estimate*base_len;
15
16     return estimate;
17 } /* Trap */

```

Program 3.3: Trap function in MPI trapezoidal rule.

the user. While we’re talking about input to parallel programs, it might be a good idea to also take a look at output. We discussed these two issues in Chapter 2. So if you remember the discussion of nondeterminism and output, you can skip ahead to Section 3.3.2.

3.3.1 Output

In both the “greetings” program and the trapezoidal rule program, we’ve assumed that process 0 can write to `stdout`, i.e., its calls to `printf` behave as we might expect. Although the MPI standard doesn’t specify which processes have access to which I/O devices, virtually all MPI implementations allow *all* the processes in `MPI_COMM_WORLD` full access to `stdout` and `stderr`. So most MPI implementations allow all processes to execute `printf` and `fprintf(stderr, ...)`.

However, most MPI implementations don’t provide any automatic scheduling of access to these devices. That is, if multiple processes are attempting to write to, say, `stdout`, the order in which the processes’ output appears will be unpredictable. Indeed, it can even happen that the output of one process will be interrupted by the output of another process.

For example, suppose we try to run an MPI program in which each process simply prints a message. (See Program 3.4.) On our cluster, if we run the program with five processes, it often produces the “expected” output:

```

Proc 0 of 5 > Does anyone have a toothpick?
Proc 1 of 5 > Does anyone have a toothpick?
Proc 2 of 5 > Does anyone have a toothpick?

```

```

#include <stdio.h>
#include <mpi.h>

int main(void) {
    int my_rank, comm_sz;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    printf("Proc %d of %d > Does anyone have a toothpick?\n",
           my_rank, comm_sz);

    MPI_Finalize();
    return 0;
} /* main */

```

Program 3.4: Each process just prints a message.

```

Proc 3 of 5 > Does anyone have a toothpick?
Proc 4 of 5 > Does anyone have a toothpick?

```

However, when we run it with six processes, the order of the output lines is unpredictable:

```

Proc 0 of 6 > Does anyone have a toothpick?
Proc 1 of 6 > Does anyone have a toothpick?
Proc 2 of 6 > Does anyone have a toothpick?
Proc 5 of 6 > Does anyone have a toothpick?
Proc 3 of 6 > Does anyone have a toothpick?
Proc 4 of 6 > Does anyone have a toothpick?

```

or

```

Proc 0 of 6 > Does anyone have a toothpick?
Proc 1 of 6 > Does anyone have a toothpick?
Proc 2 of 6 > Does anyone have a toothpick?
Proc 4 of 6 > Does anyone have a toothpick?
Proc 3 of 6 > Does anyone have a toothpick?
Proc 5 of 6 > Does anyone have a toothpick?

```

The reason this happens is that the MPI processes are “competing” for access to the shared output device, `stdout`, and it’s impossible to predict the order in which the processes’ output will be queued up. Such a competition results in **nondeterminism**. That is, the actual output will vary from one run to the next.

In any case, if we don’t want output from different processes to appear in a random order, it’s up to us to modify our program accordingly. For example, we can have each

process other than 0 send its output to process 0, and process 0 can print the output in process rank order. This is exactly what we did in the “greetings” program.

3.3.2 Input

Unlike output, most MPI implementations only allow process 0 in `MPI_COMM_WORLD` access to `stdin`. This makes sense: If multiple processes have access to `stdin`, which process should get which parts of the input data? Should process 0 get the first line? process 1 get the second? Or should process 0 get the first character?

So, to write MPI programs that can use `scanf`, we need to branch on process rank, with process 0 reading in the data, and then sending it to the other processes. For example, we might write the `Get_input` function shown in Program 3.5 for our parallel trapezoidal rule program. In this function, process 0 simply reads in the values

```

1 void Get_input(
2     int      my_rank    /* in */,
3     int      comm_sz    /* in */,
4     double*  a_p        /* out */,
5     double*  b_p        /* out */,
6     int*     n_p        /* out */) {
7     int dest;
8
9     if (my_rank == 0) {
10        printf("Enter a, b, and n\n");
11        scanf("%lf %lf %d", a_p, b_p, n_p);
12        for (dest = 1; dest < comm_sz; dest++) {
13            MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
14            MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
15            MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
16        }
17    } else { /* my_rank != 0 */
18        MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
19                MPI_STATUS_IGNORE);
20        MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
21                MPI_STATUS_IGNORE);
22        MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
23                MPI_STATUS_IGNORE);
24    }
25 } /* Get_input */

```

Program 3.5: A function for reading user input.

for *a*, *b*, and *n*, and sends all three values to each process. So this function uses the same basic communication structure as the “greetings” program, except that now process 0 is sending to each process, while the other processes are receiving.